SoulMap: A Vietnamese Astrology Question Answering System using Retrieval-Augmented Generation

My Anh Tran Nguyen
20235474
Anh.TNM235474

Huyen Nguyen Thu
20235507
Huyen.NT235507

Ha Dan Tran Le
20235483
Dan.TLH235483

Abstract

SoulMap is a Vietnamese question answering system designed for the domain of traditional astrology. Instead of relying only on the internal knowledge of a Large Language Model, the system applies Retrieval-Augmented Generation (RAG) to retrieve relevant knowledge from a custom knowledge base before generating the final answer. The dataset is collected from PDF documents, then preprocessed, normalized, and divided into two types of chunks: structured chunks based on house-star-interpretation rules and fixed-size chunks based on the context window. The system also generates a personalized astrology chart from the user's birth information and uses this chart to filter relevant documents. Gemini is used as the backbone LLM, while `bkai-foundation-models/vietnamese-bi-encoder` is used as the Vietnamese embedding-based retriever. Initial results show that the system can generate personalized summaries and answer follow-up questions based on retrieved evidence. The repository is published at: SoulMap

Keywords: RAG, NLP, Large language models.

1 Introduction

Question Answering (QA) systems have become increasingly popular in natural language processing because they allow users to access information quickly through natural language queries. However, building a QA system for a specialized domain is more challenging than building a general-purpose chatbot. In specialized domains, the system must rely on a trusted knowledge source and should avoid generating unsupported information.

This project focuses on the domain of Vietnamese traditional astrology. In this domain, knowledge is usually stored in books, PDF documents, or long textual materials. These documents are often semi-structured or unstructured, making them difficult to search and use directly in an automatic QA system.

SoulMap is developed to address this problem. The system allows users to enter personal birth information, including full name, gender, date of birth, and birth time. From this input, the system generates a personalized astrology chart. When the user asks a question, the system retrieves relevant knowledge from the processed dataset and uses a Large Language Model to generate a final answer.

The main objectives of this project are:

- To build a pipeline for generating a personalized astrology chart from user input.
- To collect, preprocess, and structure astrology knowledge from PDF documents.
- To develop a RAG-based question answering system for Vietnamese astrology.
- To combine semantic retrieval, structured filtering, and keyword matching.
- To generate answers that are grounded in retrieved context and avoid unsupported claims.

2 Related Work

2.1 Domain-specific Question Answering

Domain-specific Question Answering focuses on answering questions within a narrow knowledge domain, such as medicine, law, finance, education, or technical documentation. Unlike open-domain QA, domain-specific QA requires the system to follow a predefined knowledge source. This is important because the correctness of the answer depends not only on fluency but also on whether the answer is grounded in domain-specific evidence.

In this project, the domain is Vietnamese traditional astrology. The answer should depend on both the general astrology knowledge extracted from documents and the user's personal astrology chart. Therefore, the system needs to combine document retrieval with chart-based filtering.

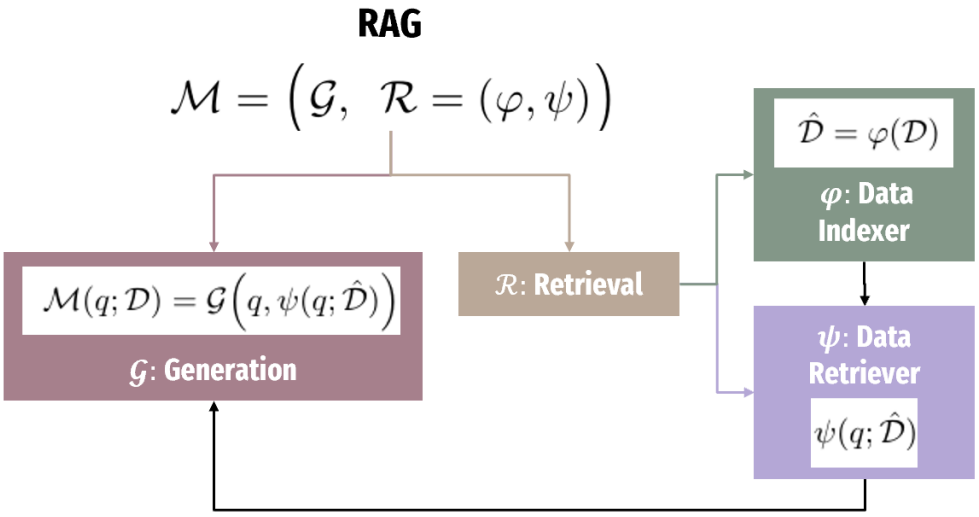

Figure 1: Overview of the Retrieval-Augmented Generation (RAG) architecture. The retriever $\mathcal{R} = (\varphi, \psi)$ consists of a data indexer φ and a data retriever ψ. Given a query q, the retriever selects relevant documents $\hat{D}$ from the corpus D, and the generator $\mathcal{G}$ produces the final output conditioned on both the query and the retrieved documents.

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation is a method that combines information retrieval and text generation. Instead of asking the LLM to answer directly, the system first retrieves relevant documents from a knowledge base. These documents are then inserted into the prompt as context for the LLM.

Figure 1 illustrates the general architecture of Retrieval-Augmented Generation (RAG), which combines external document retrieval with neural text generation (Lewis et al., 2020). A RAG model can be viewed as a combination of a generator $\mathcal{G}$ and a retriever $\mathcal{R} = (\varphi, \psi)$. The data indexer φ first encodes and organizes the external corpus D into an indexed representation $\hat{D} = \varphi(D)$. Given a user query q, the data retriever ψ searches this indexed corpus and returns relevant documents $\psi(q; \hat{D})$. The generator then produces the final response conditioned on both the query and the retrieved evidence: $\mathcal{M}(q; D) = \mathcal{G}(q, \psi(q; \hat{D}))$.

In SoulMap, this architecture is adapted to Vietnamese astrology question answering. The external corpus consists of structured and fixed-size chunks extracted from astrology documents. The retriever selects chunks related to the user's question and astrology chart, while Gemini acts as the generator that produces the final grounded answer.

RAG is particularly suitable for SoulMap because it separates domain knowledge from the language model itself. Instead of asking the LLM to generate an answer only from its parametric memory, the system first retrieves relevant evidence from the astrology knowledge base and uses it as grounded context for generation. This helps reduce hallucination, especially in a domain where unsupported interpretations may lead to misleading answers. In addition, the knowledge base can be expanded or corrected independently without retraining the LLM. This makes RAG well suited for domain-specific question answering, where the required knowledge is specialized, frequently stored in external documents, and not necessarily covered by the model's internal parameters.

2.3 Vietnamese Embedding Model

Because both the knowledge base and user questions are written in Vietnamese, the retriever requires an embedding model that can represent Vietnamese semantics effectively. SoulMap uses `bkai-foundation-models/vietnamese-bi-encoder` to encode user queries and knowledge chunks into dense vector representations.

In this space, semantically related queries and documents are expected to be close to each other, even when they do not share exactly the same surface words. This is important for Vietnamese astrology QA, where users may ask questions in informal language while the source documents use domain-specific terms such as house names, star names, and traditional interpretation phrases. The retriever then ranks candidate chunks by comparing their embedding vectors with the query vector.

3 Dataset

3.1 Data Source

The knowledge base of SoulMap is built from three Vietnamese astrology books in PDF format: *Tu Vi Thuc Hanh*(Dich Ly Huyen Co, n.d.), *Tu Vi Dau So Tan Bien*(Van Dang Thai Thu Lang, 1956), and *Tu Vi Tong Hop*(Nguyen Phat Loc, 1974). These documents cover both chart construction and interpretation knowledge. They include rules for determining the twelve houses, heavenly stems and earthly branches, destiny elements, star placement, star brightness, major periods, and interpretation methods for different houses and stars.

Among them, *Tu Vi Dau So Tan Bien* provides a relatively clear structure, including sections on chart construction, general interpretation, twelve-house interpretation, star characteristics, and period interpretation. Therefore, it is useful for extracting structured house–star interpretation rules. *Tu Vi Thuc Hanh* contains practical instructions for manually constructing a chart and placing stars, which supports the implementation of the chart generation module. *Tu Vi Tong Hop* is a more comprehensive reference that discusses the theoretical background of astrology, the role of houses and stars, and the interpretation of human life aspects.

The original PDF documents are not directly suitable for retrieval because they are long-form, semi-structured, and sometimes contain OCR or encoding noise. Therefore, we first extract raw text from the PDFs and then convert the extracted content into retrievable data. The processed dataset is organized into two forms: structured chunks and fixed-size chunks. Structured chunks are created from parsed house–star rules, while fixed-size chunks preserve broader textual context from the original documents.

3.2 Preprocessing

The preprocessing pipeline includes the following steps:

1. **PDF text extraction**: The text content is extracted from PDF documents.

2. **Text cleaning**: Redundant spaces, noisy symbols, and formatting errors are removed.

3. **Normalization**: House names, star names, topics, and conditions are normalized into consistent identifiers.

4. **Parsing**: Regular-expression-based scripts are used to extract houses, stars, rules, and interpretations.

5. **Serialization**: The processed data is saved into JSON and JSONL files for retrieval.

Normalization is important because the same house or star may appear in different text formats. For example, house names are normalized into IDs such as `cung_menh`, `cung_phu_mau`, and `cung_thien_di`. This allows the retriever to match documents with the user's chart more reliably.

3.3 Fixed-size Chunking

Fixed-size chunking is the first chunking strategy used in SoulMap. In this approach, the extracted text from the PDF documents is split into passages with a predefined length. Each passage is then stored as a retrievable document and used by the dense retriever. This strategy follows the common retrieval setting in RAG-based systems, where long documents are divided into smaller passages before retrieval (Lewis et al., 2020; Karpukhin et al., 2020).

Fixed-size chunks are useful because the original PDF documents are long, semi-structured, and sometimes noisy. Some parts of the documents contain continuous explanations, tables, examples, or discussions that are difficult to convert directly into structured rules. By keeping these parts as fixed-size chunks, the system can still retrieve useful background information even when rule-based parsing is incomplete.

The chunk size is selected according to the context window of the LLM. Each chunk should be short enough to fit into the prompt together with other retrieved chunks, but long enough to preserve meaningful context. In the retrieval stage, the user query is encoded into an embedding vector and compared with the embeddings of fixed-size chunks. The top-ranked chunks are then used as evidence for answer generation.

The main advantage of fixed-size chunking is high recall. It allows the system to retrieve relevant information from any part of the source documents, including sections that are not yet parsed into structured data. It is also simple to implement and does not require strong assumptions about the document format.

However, fixed-size chunking also has several limitations. First, a chunk may cut across logical

boundaries, causing an interpretation rule to be split into different passages. Second, it does not explicitly represent important astrology fields such as house, star, condition, or required stars. As a result, fixed-size chunks are less effective for chart-specific filtering. A retrieved chunk may be semantically related to the query but not actually applicable to the user's astrology chart.

3.4 Structured Chunking

To address the limitations of fixed-size chunking, SoulMap also uses structured chunking. Structured chunking converts astrology interpretation rules into documents with explicit fields. Instead of treating each passage as plain text, the system extracts the internal structure of the astrology knowledge, such as topic, house, star, condition, required stars, and interpretation.

Structured chunking is necessary because Vietnamese astrology interpretation is highly conditional. A rule is usually valid only when a specific star appears in a specific house, or when a combination of stars satisfies certain conditions. Therefore, semantic similarity alone is not sufficient. The retriever must also know whether the retrieved rule matches the user's generated chart.

Each structured chunk is designed to represent meaningful interpretation rule, allows the retriever to filter documents according to the user's chart before ranking them. For example, if a rule is about a specific star in the `cung_menh`, the system can check whether that star actually appears in the user's `Cung_menh`. If the condition is not satisfied, the rule can be removed from the candidate set or assigned lower priority. This makes retrieval more personalized and reduces irrelevant evidence.

The main advantage of structured chunking is precision. It makes domain-specific constraints explicit and helps the system retrieve evidence that is not only semantically related to the question but also compatible with the user's chart. It also makes the retrieved evidence easier to insert into prompts because each chunk already contains a clear condition and interpretation.

The main limitation is that structured chunking requires more preprocessing effort. Since the source PDFs have inconsistent formatting and may contain OCR errors, rule extraction is not always reliable. Some rules may be missed or incorrectly parsed. For this reason, SoulMap combines structured chunks with fixed-size chunks. Fixed-size chunks improve recall, while structured chunks improve chart-specific precision.

3.5 Comparison of Chunking Strategies

Table 1 summarizes the differences between the two chunking strategies used in SoulMap.

Aspect	Fixed-size chunks	Structured chunks
Main goal	Preserve broad textual context	Represent domain-specific rules
Input	Raw extracted PDF text	Parsed house–star interpretations
Strength	Higher recall and easy construction	Higher precision and chart filtering
Weakness	May ignore logical boundaries	Requires reliable parsing
Best for	General explanations and noisy text	Personalized astrology reasoning

Table 1: Comparison between fixed-size and structured chunking in SoulMap.

3.6 Structured Data Processing Pipeline

The original knowledge source of SoulMap is a set of Vietnamese astrology PDF documents. These PDFs contain long-form explanations about stars, houses, star combinations, chart construction, and interpretation rules. However, raw PDF content is not directly suitable for retrieval because it is unstructured, long, and may contain formatting or encoding noise. Therefore, we design a multi-stage data processing pipeline to gradually transform the raw documents into structured retrieval units.

3.6.1 From Raw PDF to Plain Text

In the first stage, the PDF documents are converted into plain text files. This step extracts the textual content from the original books and stores it in a simpler format such as `text.txt`. The purpose of this stage is to remove the dependency on PDF layout and make the content easier to clean, normalize, and parse.

After extraction, the text is preprocessed by fixing encoding issues, removing unnecessary whitespace, and normalizing Vietnamese terms. This step is important because the same house or star may appear in different forms due to accents, spacing, or OCR noise. For example, star and house names are later normalized into identifiers such as `tu_vi` and `cung_menh`.

3.6.2 House-based Interpretation Data

After obtaining clean text, the system parses the content into house-based interpretation files. Each house contains a set of stars, and each star may

have different interpretation contexts such as general meaning, male chart, female chart, or traditional verses. For example, the house interpretation data stores fields such as `star_id`, `context_type`, and `rules`. A rule can contain one or more interpretation texts for a specific star in a specific house.

This representation is useful because Vietnamese astrology interpretation is highly conditional. A statement is usually not universal; it depends on where a star appears, which house it belongs to, and what contextual condition is being considered. Therefore, house-based parsing helps the system preserve the logical structure of the original documents instead of treating the whole book as plain text only.

3.6.3 Star Dictionary

In addition to house-based rules, the system also builds a star dictionary. This dictionary stores the basic meaning of each star, including its name, type, element, polarity, direction, core characteristics, brightness positions, and full descriptive content. For example, a star entry may include fields such as `star_id`, `name`, `type`, `element`, `core_characteristics`, and `positions`.

The star dictionary is used as a lookup resource. When a user query contains a specialized astrology term, such as a main star or auxiliary star, the system can use this dictionary to understand what the term means. This is especially useful when the language model does not have enough reliable knowledge about a specific Vietnamese astrology concept. In this case, the dictionary acts as an internal domain lexicon that supports retrieval and prompt grounding.

3.6.4 Topic Mapping

The system also constructs a topic mapping between life aspects and related houses. In Vietnamese astrology, a user question about a topic is usually associated with one or more houses. For example, questions about career are related to houses such as `Cung_menh` and `Cung_quan_loc`, while questions about finance are related to houses such as `Cung_tai_bach`, `Cung_dien_trach`, and `Cung_phuc_duc`.

This mapping helps the retriever narrow the search space when the user asks a topic-level question. If the query is about study, career, finance, health, family, relationship, or travel, the system can prioritize chunks from the houses that are most relevant to that topic. Therefore, topic mapping connects natural language user questions with the symbolic structure of the astrology chart.

3.6.5 Final RAG Documents

In the final stage, the processed resources are converted into `rag_documents`. Each RAG document is a retrieval-ready unit that combines structured metadata and natural language interpretation. A typical RAG document contains the following fields:

- `id`: a unique identifier of the document;
- `topic`: one or more related life aspects;
- `house_id`: the house associated with the rule;
- `house_name`: the normalized house name;
- `star_id`: the star mentioned in the rule;
- `required_stars`: additional star conditions if available;
- `context_type`: the interpretation context;
- `condition`: the condition under which the rule is applied;
- `interpretation`: the original interpretation text;
- `chunk_text`: the final text used for retrieval and prompting.

The `chunk_text` field is built from the structured fields and the original interpretation. It usually contains the topic, house, condition, and interpretation in a readable textual format. This allows the retriever to use both semantic similarity and structured metadata during search.

Overall, the data processing pipeline transforms raw PDF documents into a domain-aware knowledge base. The plain text files preserve the original content, the house-based data captures conditional interpretation rules, the star dictionary provides term definitions, the topic mapping links user questions to relevant houses, and the final RAG documents combine all these signals into retrievable chunks for the question answering system.

4 Method

We follow the general RAG architecture shown in Figure 1. In our system, φ corresponds to the indexing process over astrology chunks, ψ corresponds to the hybrid retriever, and $\mathcal{G}$ corresponds to Gemini.

4.1 System Overview

SoulMap consists of two main phases: user chart initialization and question answering. In the initialization phase, the user provides personal information such as full name, gender, date of birth, and birth time. The system converts the solar date into a lunar date, computes astrology-related attributes, and generates a full astrology chart with twelve houses.

In the question answering phase, the user submits a natural language question. The system builds a chart index, retrieves relevant chunks from the knowledge base, constructs a grounded prompt, and sends the prompt to Gemini to generate the final answer.

A more detailed end-to-end illustration of this pipeline, from user input and chart generation to retrieval, prompt construction, Gemini generation, and chat UI display, is presented in Figure 5 in the Demo section.

4.2 User Chart Generation

The system first converts the user's solar birth date into a lunar date. Then it calculates information such as the heavenly stem and earthly branch of the birth year, destiny element, astrology configuration, main house, life master, body master, and body house.

After that, the system creates twelve houses in the following order:

Mệnh, Phụ Mẫu, Phúc Đức, Điền Trạch, Quan Lộc, Nô Bộc, Thiên Di, Tật Ách, Tài Bạch, Tử Tức, Phu Thê, Huynh Đệ.

Each house stores house name, zodiac sign, main stars, auxiliary stars, brightness levels, life-cycle stage, Tuần–Triệt markers, and major period information.

4.3 Chart Index

After generating the chart, SoulMap builds a chart index. This index maps each house to its stars and related attributes. It is used by the retriever to check whether a structured document is compatible with the user's chart.

For example, if a document discusses the star `tu_vi` in `cung_menh`, the system checks whether the user's `cung_menh` actually contains `tu_vi`. If the condition is not satisfied, the document can be filtered out or assigned lower priority.

4.4 Retriever Design

The retriever is responsible for selecting the most relevant evidence before the final prompt is sent to the LLM. This step is essential in RAG systems because the generator should answer based on retrieved external knowledge rather than relying only on its internal parameters (Lewis et al., 2020). In SoulMap, retrieval is more challenging than ordinary document retrieval because the relevance of a chunk depends on both the user question and the user's generated astrology chart.

The knowledge base contains two document types. The first type is *fixed-size chunks*, which are produced by splitting the original PDF text into short passages. These chunks preserve broad textual context and are useful for general explanations. The second type is *structured chunks*, which are produced from parsed astrology rules. Each structured chunk contains metadata such as topic, house, star, required stars, condition, and interpretation. This metadata allows the system to perform chart-based filtering before ranking documents.

To provide a visual overview and make this retrieval process clearer, Figure 2 summarizes the main data flow from the knowledge base and chart filter to hybrid scoring and top-k evidence selection.

4.4.1 BM25 Sparse Retrieval

As an initial retrieval baseline, SoulMap uses keyword-based retrieval inspired by BM25. BM25 is a sparse retrieval method that scores a document according to term frequency, inverse document frequency, and document length normalization (Robertson and Zaragoza, 2009). This approach is useful when the query contains explicit domain keywords, such as a house name, star name, or topic.

The BM25 score can be written as:

$$S_{\mathrm{BM25}}(q,d) = \sum_{t \in q} \mathrm{IDF}(t) \frac{f(t,d)(k_1+1)}{f(t,d) + k_1\left(1 - b + b\frac{|d|}{\mathrm{avgdl}}\right)}. \tag{1}$$

Here, $f(t,d)$ is the frequency of term t in document d, $|d|$ is the document length, avgdl is the

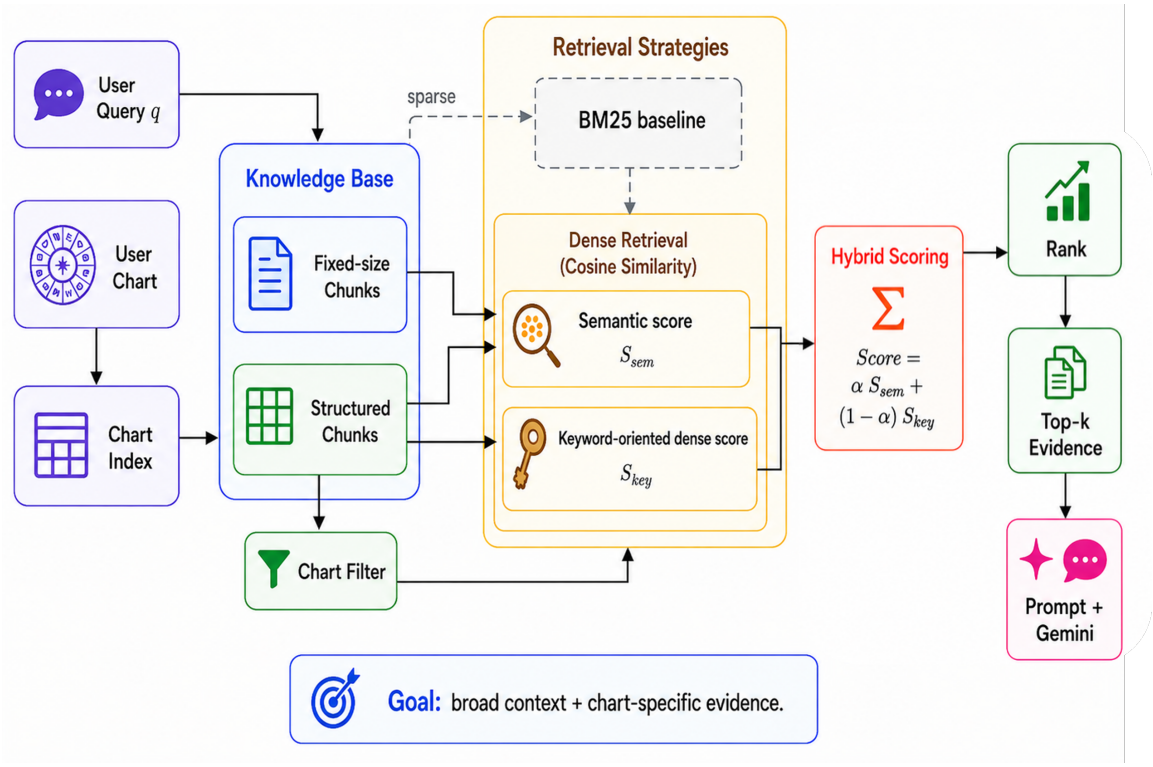

Figure 2: Overview of the SoulMap retriever design. The retriever combines fixed-size chunks, structured chunks, chart-based filtering, sparse BM25 retrieval, and dense retrieval with cosine similarity. The semantic and keyword-oriented scores are merged through hybrid scoring before ranking the top-k evidence for prompt construction.

average document length, and k_1 and b are BM25 hyperparameters.

However, sparse retrieval has a limitation: it relies heavily on lexical overlap. For example, a query about "career direction" may be semantically related to documents about `quan_loc`, but the exact words may not match. Therefore, SoulMap later moves toward dense retrieval based on embedding similarity.

4.4.2 Dense Retrieval with Cosine Similarity

In the dense retrieval stage, the user query and candidate documents are encoded into embedding vectors. Dense retrieval has been widely used in open-domain QA because it can capture semantic similarity beyond exact word matching (Karpukhin et al., 2020).

The semantic score is computed as:

$$S_{\text{sem}}(q, d) = \mathbf{e}_q \cdot \mathbf{e}_d. \tag{2}$$

where $\mathbf{e}_q$ is the query embedding and $\mathbf{e}_d$ is the document embedding. Since embeddings are normalized, the dot product is equivalent to cosine similarity.

4.4.3 Keyword Representation with Dense Similarity

Besides using the full `chunk_text`, SoulMap also builds a compact keyword representation from structured metadata:

$$K(d) = \text{concat}(f_1, f_2, \ldots, f_n). \tag{3}$$

where f_i denotes a structured field such as `topic`, `house_id`, `star_id`, `required_stars`, `context_type`, or `condition`.

Instead of relying only on sparse keyword matching, the final version also encodes this keyword representation into an embedding vector. The keyword-oriented dense score is computed as:

$$S_{\text{key}}(q, d) = \mathbf{e}_q \cdot \mathbf{e}_{K(d)}. \tag{4}$$

This allows the retriever to preserve the benefit of structured metadata while still using semantic similarity.

4.5 Prompt Construction

Prompt construction connects the retriever and the LLM. Instead of sending the raw user question directly to Gemini, SoulMap constructs a grounded prompt from multiple sources: the user's astrology

chart, the retrieved evidence, the initial interpretation, and recent conversation context.

The prompt design follows common prompting strategies, including role prompting, instruction prompting, context grounding, and retrieval-augmented prompting (Brown et al., 2020; Liu et al., 2023; Ouyang et al., 2022; Lewis et al., 2020). The main goal is to make the generated answer relevant to the question, grounded in the retrieved evidence, and consistent with the user's chart.

4.5.1 Prompting Techniques

The prompt design follows common prompting strategies studied in prior work, including prompt-based learning, instruction following, and retrieval-augmented generation (Brown et al., 2020; Liu et al., 2023; Ouyang et al., 2022; Lewis et al., 2020). Table 2 summarizes how these techniques are used in SoulMap.

Technique	Usage
Role prompting	Defines the model role for domain-specific answering (Liu et al., 2023).
Context grounding	Provides chart data and external evidence to guide generation (Lewis et al., 2020).
Instruction prompting	Specifies constraints such as groundedness and safe answering (Ouyang et al., 2022).
Output formatting	Uses explicit textual instructions to control answer structure (Brown et al., 2020; Liu et al., 2023).
Conversation memory	Includes recent dialogue context to maintain coherent follow-up answers (Brown et al., 2020).
RAG prompting	Injects retrieved documents into the prompt before generation (Lewis et al., 2020).

Table 2: Prompting techniques used in SoulMap and their related prior work.

4.5.2 Advantages and Limitations

The prompt construction design has several advantages. First, it separates the user's raw question from the final LLM input by adding structured chart data and retrieved context. Second, it improves answer grounding by requiring the model to use the provided evidence. Third, it supports different interaction stages, including initial interpretation and follow-up question answering. However, there are still some limitations. The chart data can be long, which increases the number of input tokens. The current prompt does not always require explicit citations from retrieved documents. In addition, if the retriever returns irrelevant chunks, the generated answer may still become less accurate. Future work should focus on shortening chart context, adding evidence citation, and integrating intent-specific prompt templates more deeply into the RAG pipeline.

Overall, prompt construction in SoulMap plays the role of connecting user data, retrieved knowledge, and the LLM. By combining astrology chart information, RAG context, and clear instructions, the system can generate personalized answers that are more grounded and suitable for the astrology question answering task.

5 Experiments

5.1 Retrieval Modes

The system supports five retrieval modes through the `retrieval-mode` parameter. These modes are used to compare different retrieval sources and scoring strategies. Table 3 summarizes the retrieval modes used in our experiments.

Different retrieval modes are useful for different purposes:

- `fixed_similarity` is used as a simple baseline over long-form text chunks.
- `fixed_structured_similarity` is used when both general document context and chart-compatible structured rules are needed.
- `fixed_structured_keyword` is used as the main mode for the final QA system because it combines recall from fixed-size chunks and precision from structured metadata.
- `structured_similarity` is used when the system only needs chart-filtered structured rules.
- `structured_keyword` is useful when the query contains explicit house names, star names, topics, or rule conditions.

Overall, this hybrid retrieval design helps SoulMap retrieve both broad explanations from the original PDF sources and precise chart-specific interpretations from structured astrology rules.

5.1.1 Score Combination

In the default mode, `fixed_structured_keyword`, each candidate document receives both a semantic score and a keyword score. The final

Mode	Data source	Description
`fixed_similarity`	Fixed-size chunks	Performs semantic retrieval only over fixed-size chunks. This mode is used as a simple dense retrieval baseline over long-form text.
`fixed_structured_sim`	Fixed-size chunks + structured chunks	Combines fixed-size chunks with structured documents. Structured documents are filtered using chart information and then ranked by semantic similarity.
`fixed_structured_key`	Fixed-size chunks + structured chunks	The default mode. It combines semantic similarity over textual content with keyword-oriented dense similarity over structured metadata.
`structured_similarity`	Structured chunks	Uses only structured chunks. Documents are filtered according to the user's chart and then ranked by semantic similarity over `chunk_text`.
`structured_keyword`	Structured chunks	Uses only structured chunks and ranks documents mainly through structured fields such as `topic`, `house_id`, `star_id`, `required_stars`, and `condition`.

Table 3: Retrieval modes supported by the SoulMap retriever.

retrieval score is computed by a weighted interpolation:

$$\mathrm{Score}(d, q) = \alpha S_{\mathrm{sem}}(q, d) + (1 - \alpha) S_{\mathrm{key}}(q, d). \tag{5}$$

Here, d is a candidate document, q is the user query, $S_{\mathrm{sem}}(q, d)$ is the semantic similarity score, $S_{\mathrm{key}}(q, d)$ is the keyword-based score, and α is the interpolation weight. A larger α gives more importance to semantic similarity, while a smaller α gives more importance to structured keyword signals. In our experiments, we set $\alpha = 0.75$. This setting gives higher priority to semantic similarity while still preserving the effect of structured metadata. After scoring, candidate documents are sorted in descending order, and the top-k documents are inserted into the prompt as external evidence for Gemini.

5.1.2 Chart-based Filtering

Before ranking structured chunks, the system builds a chart index from the user's generated chart. This index stores the house names, zodiac signs, main stars, auxiliary stars, brightness levels, and other attributes of each house. For a structured document, the retriever checks whether its house and star conditions match the user's chart. For example, if a document is about a specific star in the `Cung_menh`, the system verifies whether that star actually appears in the user's `Cung_menh`. Documents that satisfy the chart conditions are kept or prioritized, while irrelevant structured rules can be removed from the candidate set. If no structured document matches the chart, the system falls back to the full structured collection to avoid returning an empty result.

5.1.3 Retriever Evolution

We first implemented a sparse keyword retriever based on BM25 to test whether explicit metadata fields such as topic, house, and star names could support domain-specific retrieval. This baseline is simple and effective when the user query contains exact domain terms. However, it is less effective when the query uses different wording from the source documents.

To improve semantic matching, we then moved to dense retrieval. In the dense setting, queries and documents are encoded into normalized embeddings, and similarity is computed using cosine similarity. This allows the retriever to match semantically related questions and documents even when they do not share the same surface words.

The final retriever combines both ideas. It uses dense similarity over full document content and dense similarity over structured keyword representations. This design keeps the semantic flexibility of dense retrieval while preserving the domain constraints encoded in structured metadata.

5.1.4 Score Combination

In the default mode, `fixed_structured_keyword`, each candidate document receives both a semantic score and a keyword score. The final retrieval score is computed by a weighted interpolation:

$$\mathrm{Score}(d, q) = \alpha S_{\mathrm{sem}}(q, d) + (1 - \alpha) S_{\mathrm{key}}(q, d). \tag{6}$$

Here, d is a candidate document, q is the user query, $S_{\mathrm{sem}}(q, d)$ is the semantic similarity score, $S_{\mathrm{key}}(q, d)$ is the keyword-based score, and α is the

interpolation weight. A larger α gives more importance to semantic similarity, while a smaller α gives more importance to structured keyword signals. In our experiments, we set $\alpha = 0.75$. This setting gives higher priority to semantic similarity while still preserving the effect of structured metadata. After scoring, candidate documents are sorted in descending order, and the top-k documents are inserted into the prompt as external evidence for Gemini.

5.2 Prompt Construction

Prompt construction is an important component of the SoulMap system. It is used to build the input for the backbone LLM, Gemini, so that the model can generate personalized astrology interpretations and answer follow-up questions from users. Instead of sending the user's question directly to the LLM, the system constructs a prompt from multiple sources of information, including the user's astrology chart, retrieved knowledge from the RAG module, the initial interpretation, and the recent conversation context. The main goal of prompt construction is to make the generated answer grounded, relevant, and consistent with the available evidence. Since astrology interpretation is a domain-specific task, the prompt must clearly define the role of the model, provide sufficient context, and restrict the model from making unsupported or absolute claims.

5.2.1 Initial Prompt

The initial prompt is used after the chart has been generated. It contains the chart information and retrieved documents related to important chart features. The model is asked to summarize the user's main tendencies, including personality, study or career, finance, relationship, and important aspects to balance.

The initial prompt also includes constraints: the model should not make absolute predictions, should not invent information outside the provided context, and should present the answer as a reference.

5.2.2 Follow-up Prompt

The follow-up prompt is used when the user asks a specific question after receiving the initial interpretation. It contains the current user query, the astrology chart, the initial summary, recent conversation context, and newly retrieved documents.

The model is instructed to answer the question directly and clearly. If the retrieved context is insufficient, the model should state that there is not enough evidence instead of hallucinating an answer.

5.2.3 Role of RAG in Prompt Construction

The prompt is not static. Before constructing the final prompt, the system retrieves relevant knowledge from the knowledge base using the RAG module. For the initial interpretation, the retriever selects important documents related to the user's chart. For follow-up questions, the retriever searches for documents related to both the user's question and chart features. The retrieved documents are inserted into the prompt as supporting context. This helps the LLM generate answers based on external evidence rather than relying only on its internal knowledge. As a result, the system can reduce hallucination and produce answers that are more relevant to the user's chart.

5.2.4 Conversation Context

To maintain continuity in the dialogue, the system also includes recent conversation history in the prompt. However, only a limited number of recent messages are used. This design helps preserve important context while avoiding overly long prompts and reducing token cost. The conversation context allows the model to understand what has already been discussed and generate more coherent follow-up responses. For example, if the user asks 'What about my career?" after a previous discussion about study orientation, the model can use the recent context to interpret the question more accurately.

5.2.5 Intent-aware Prompting

The system is designed to support intent-aware prompting. User questions can be grouped into several major categories, such as relationship, career and finance, education, long-term strategy, personal growth, and general interpretation. Each category can have a specialized prompt template that focuses on the most relevant houses, stars, and interpretation rules. Although the current system mainly uses a general RAG-based prompt, intent-aware prompting provides a clear direction for future improvement. It can help the system generate more focused answers depending on the user's question type.

5.3 Experimental Setup

Table 4 summarizes the main experimental configuration of SoulMap. The system follows the

retrieval-augmented generation framework (Lewis et al., 2020), uses an instruction-following LLM for generation (Ouyang et al., 2022), and applies dense retrieval with a bi-encoder architecture (Karpukhin et al., 2020).

Component	Configuration
Task	Domain-specific QA
Domain	Vietnamese astrology
Method	RAG (Lewis et al., 2020)
Backbone LLM	Gemini-2.5-flash
Retriever encoder	BKAI Vietnamese bi-encoder
Retriever type	Hybrid semantic–keyword
Retrieval data	Structured + fixed chunks
Top-k	8
α	0.75
Input context	User chart + retrieved chunks
Output language	Vietnamese

Table 4: Main experimental configuration used in SoulMap.

5.4 Evaluation Dataset

To evaluate the system in a controlled and reproducible manner, we construct a synthetic evaluation dataset consisting of 100 users and 108 questions.

Each user is assigned a generated astrological chart based on demographic attributes (gender and date of birth). This allows consistent simulation of user-specific structured contexts.

The evaluation questions are designed to cover multiple difficulty levels and diverse combinations of astrological "houses". In total, the dataset includes:

Difficulty	# Questions	# Relevant Features
Easy	48	1
Medium	36	2
Hard	24	3–4
Total	**108**	–

Table 5: Distribution of evaluation questions by difficulty and number of relevant astrological houses.

Each question is annotated with a set of expected features, which are used for retrieval-level evaluation.

5.5 Feature Extraction

To enable evaluation of both retrieval and generation, we define a unified feature extraction pipeline.

We extract two types of features:

- **Structured features**: extracted from structured RAG documents, including house identifiers, star identifiers, and required-star relationships.
- **Unstructured features**: extracted from fixed-size text chunks using a vocabulary-based mapping. The vocabulary is constrained to a closed set of canonical symbols consisting only of house identifiers and star identifiers, ensuring that both structured and unstructured inputs are projected into the same symbolic feature space.

We apply the same feature extraction pipeline to generated answers as used for unstructured retrieval chunks. This ensures consistency between retrieval and generation evaluation spaces.

5.6 Evaluation Metrics

5.6.1 Retrieval Evaluation

We evaluate the retrieval component independently before evaluating generation. The goal is to measure whether the retriever returns documents containing the necessary semantic and structured features required to answer a query.

Retrieval quality is measured using a feature-based evaluation framework. Each query is associated with a set of expected features it is relevant to. Retrieved documents are processed into feature sets according to their type (structured or unstructured).

We define the following metrics:

- **Feature Recall**: proportion of expected features that appear in the retrieved set.
- **Query Success**: whether all expected features are fully covered by the retrieved documents.
- **Missing Features**: set difference between expected and retrieved features.
- **Hit@K**: whether at least one expected feature appears in any of the top-k retrieved documents after feature extraction.
- **Mean Reciprocal Rank (MRR)**: inverse rank of the first retrieved document whose extracted feature set overlaps with any expected feature.

Formally, feature recall is defined as:

$$\text{Feature Recall} = \frac{|F_{expected} \cap F_{retrieved}|}{|F_{expected}|} \quad (7)$$

where $F_{expected}$ is the set of annotated features for a query and $F_{retrieved}$ is the union of features extracted from retrieved documents.

5.6.2 Answer Generation Evaluation

To evaluate the quality of generated responses, we assess three main aspects: fluency, grounding, and factual consistency with respect to retrieved context and structured astrological knowledge.

Unlike retrieval evaluation, which focuses on document relevance, generation evaluation measures whether the final answer is coherent, supported by evidence, and consistent with the user's astrological chart.

To reduce computational cost, we evaluate a subset of 10 users. For each selected user, we sample 12 questions using a deterministic stride-based strategy over the ordered question indices with stride 9, ensuring uniform coverage across the question space while maintaining reproducibility.

Fluency Evaluation We evaluate response fluency using perplexity (PPL), a standard language-model-based measure of linguistic naturalness. Perplexity quantifies how well a language model predicts a sequence of tokens, with lower values indicating more fluent and coherent text.

For each generated response, the text is tokenized using the tokenizer associated with the evaluation language model. Let $x_1, x_2, \ldots, x_n$ denote the resulting token sequence. We compute the negative log-likelihood of the sequence using a causal language model and convert it to perplexity as

$$\mathrm{PPL}(x) = \exp\left(-\frac{1}{n}\sum_{i=1}^{n}\log p(x_i \mid x_{<i})\right) \quad (8)$$

where $p(x_i \mid x_{<i})$ is the probability assigned by the language model to token x_i given the preceding context.

In practice, we use the GPT-2 language model as the evaluator. The entire generated response is processed in a single forward pass with teacher forcing, where the input tokens are also used as prediction targets. The model returns the average token-level cross-entropy loss over the sequence, and perplexity is computed as the exponential of this loss. Responses are truncated to a maximum length of 512 tokens during evaluation.

For each response, we report the resulting perplexity score. Lower perplexity values indicate that the generated text is more predictable under the language model and therefore generally more fluent and linguistically coherent.

We use GPT-2 as a reference language model to provide a relative measure of fluency across generation modes. Since all evaluated systems generate Vietnamese text under the same conditions, perplexity is used for comparative analysis rather than as an absolute estimate of Vietnamese language quality.

Grounding Evaluation

We evaluate grounding along three complementary dimensions:

- **Coverage**: measures how well the answer covers structured features extracted from retrieved structured documents (e.g., houses and stars). It computes the proportion of these retrieved structured features that appear in the generated answer.
- **Strict Grounding**: evaluates the proportion of extracted answer features that can be directly matched to features derived from retrieved documents (both structured and chunk-based). It is a feature-level hallucination metric measuring how much of the generated content is supported by retrieved context.
- **Prompt Grounding**: evaluates whether generated answer features are supported by the full set of available conditioning information, including structured astrological chart data, initial summary context, and retrieved document features. It measures how consistent the answer is with the complete input context provided to the model.

The coverage score is defined as:

$$Coverage = \frac{|F_{mentioned} \cap F_{retrieved}|}{|F_{retrieved}|} \quad (9)$$

where $F_{retrieved}$ denotes features extracted from retrieved documents, and $F_{mentioned}$ denotes features detected in the generated answer.

Coverage is computed in a structured feature space derived from the structured subset of retrieved documents, specifically those containing structured identifiers such as houses names or star identifiers. These documents are converted into canonical symbolic features representing houses and stars. Generated answers are mapped into

the same feature space using a shared vocabulary-based extractor, enabling direct set comparison. Unstructured fixed-size chunks are excluded from coverage computation, as they do not provide stable or reliably canonicalizable symbolic features.

5.7 Result

5.7.1 Retrieval Results

We evaluate five retrieval configurations, including: *fixed similarity*, *fixed structured similarity*, *fixed structured keyword*, *structured similarity*, and *structured keyword*. All methods are evaluated using feature-based metrics: Feature Recall, Query Success, Hit@K, and MRR.

a, Overall Performance

Table 6 summarizes the aggregated results across all users and questions.

Overall, hybrid retrieval configurations (combining structured and fixed chunks) achieve the highest feature recall, indicating that combining semantic and keyword-based signals improves coverage of relevant astrological features.

Interestingly, hybrid retrieval improves recall but not ranking quality, suggesting a trade-off between coverage and precision.

Among all settings, structured similarity-based methods outperform fixed similarity baselines on ranking-based metrics such as Hit@K and MRR, while maintaining comparable feature recall. Among all configurations, *structured similarity* achieves the strongest retrieval ranking performance, reaching a Hit@3 of 0.59 and the highest MRR (0.53).

In contrast, purely keyword-based structured retrieval performs significantly worse across all metrics, indicating that lexical matching alone is insufficient for capturing the semantic variability in astrological queries.

b, Performance by Difficulty

We analyze retrieval performance across three difficulty levels (easy, medium, hard) to understand how different retrieval strategies behave under varying query complexity.

Feature Recall

Figure 3 shows that retrieval performance is strongly dependent on query complexity.

For easy queries, most methods achieve relatively high feature recall (0.61–0.65 for the top-performing systems), indicating that simple feature compositions can be reliably captured across retrieval strategies. However, keyword-only structured retrieval underperforms even in this setting.

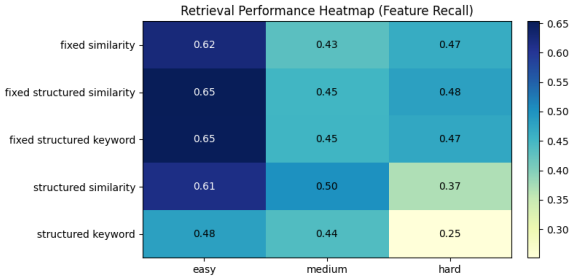

Figure 3: Feature Recall Retrieval Performance Across Modes and Difficulty Levels

For medium difficulty queries, performance decreases across all methods, with textitstructured similarity achieving the best recall (0.5017). This suggests that semantic modeling of structured attributes provides advantages in moderately complex cases involving multiple feature interactions.

For hard queries, we observe a clear divergence between methods. Fixed and hybrid retrieval variants maintain relatively stable performance (0.47–0.48), while fully structured methods degrade significantly, particularly structured keyword retrieval (0.2516). This suggests that overly strict structured matching may reduce robustness when queries involve sparse or complex feature combinations.

Hit@3

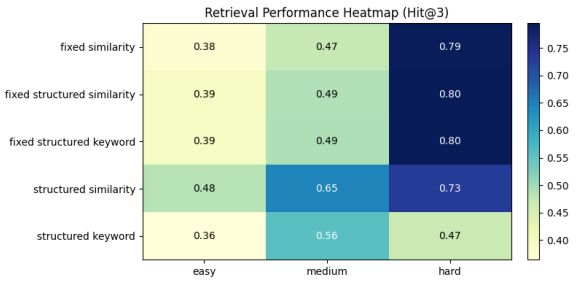

Figure 4: Hit@3 Retrieval Performance Across Modes and Difficulty Levels

From Figure 4, we evaluate ranking quality using Hit@3 across difficulty levels. For easy queries, structured similarity achieves the best performance (0.4850), indicating that semantic modeling of structured attributes improves early-stage retrieval even in simple cases.

For medium difficulty queries, structured similarity demonstrates a substantial improvement over other methods, reaching 0.6456, suggesting that semantic representation is particularly effective for

Method	Recall	Success	MRR	Hit@1	Hit@3
Fixed Similarity	0.525	0.352	0.472	0.380	0.500
Fixed + Struct Sim	0.548	0.365	0.483	0.384	0.514
Fixed + Struct Keyword	0.546	0.363	0.483	0.383	0.515
Structured Similarity	**0.522**	0.354	**0.528**	**0.436**	**0.593**
Structured Keyword	0.416	0.270	0.421	0.352	0.452

Table 6: Retrieval performance across different retrieval configurations.

multi-feature queries requiring aggregation across multiple astrological attributes.

For hard queries, we observe a convergence among most retrieval methods, with fixed-based and hybrid variants achieving approximately 0.79 Hit@3. This suggests that top-3 retrieval is relatively robust across most configurations. However, structured keyword retrieval significantly underperforms (0.4667), indicating limitations in handling sparse or complex feature compositions.

c, Discussion

Based on the empirical evidence across both overall metrics and difficulty tiers, we identify *structured similarity* and *fixed structured similarity* as the two optimal, complementary retrieval configurations. Each excels at a different frontier of the retrieval trade-off:

- ***Structured Similarity*:** This mode achieves the highest overall ranking quality across the entire dataset, securing a top MRR of 0.53, a Hit@1 of 43.6%, and a Hit@3 of 59.3%. It is particularly dominant in medium-difficulty queries, where semantic modeling of structured astrological attributes allows it to surface highly precise matches at the very top of the retrieved list.

- ***Fixed Structured Similarity*:** This configuration provides the highest global coverage, leading in overall Feature Recall (0.548) and Query Success (0.365). More importantly, it demonstrates superior resilience against hard queries. While purely structured methods degrade significantly when faced with complex, sparse feature combinations, the hybrid nature of *fixed structured similarity* maintains a stable recall (0.48) and delivers exceptional top-ranking performance (0.71 MRR and 79.5% Hit@3) under high query complexity.

Conversely, keyword-based variants (especially *structured keyword*) consistently underperformed across all difficulty tiers, proving that lexical matching alone cannot resolve the semantic nuances of astrology queries.

Consequently, we select *structured similarity* (for its optimized precision and ranking) and *fixed structured similarity* (for its robust recall and hard-query stability) as our two candidate retrieval configurations. In the next section, we carry forward these two configurations to evaluate how their distinct retrieval strengths impact downstream answer generation quality.

5.7.2 Answer Generation Results

Table 7 presents the downstream generation metrics for the two top-performing retrieval configurations. The empirical results show a clear, decisive advantage for the hybrid retrieval strategy.

Perplexity (PPL). Both models exhibit similar perplexity values (17.48 vs. 17.70), indicating comparable fluency and language modeling quality. The slight increase in PPL for the fixed structured variant suggests a marginal reduction in fluency, though the difference is not substantial.

Prompt Grounding. Both methods achieve near-perfect prompt grounding (0.996 vs. 0.998), indicating strong consistency with the full conditioning context, including structured inputs and retrieved content. The difference is negligible.

Strict Grounding. Fixed structured similarity shows a modest improvement in strict grounding (0.422 vs. 0.400), indicating a slightly higher proportion of answer features being supported by retrieved evidence. This suggests improved factual alignment at the feature level.

Coverage. The most significant difference is observed in coverage, where fixed structured similarity substantially outperforms structured similarity (0.829 vs. 0.475). This indicates that the fixed variant more effectively incorporates retrieved structured features into the generated output, leading to higher recall of relevant symbolic information.

Metric	Structured Similarity	Fixed Structured Similarity	Delta
PPL	17.48	17.70	+0.23
Coverage	0.475	0.829	+0.354
Strict Grounding	0.400	0.422	+0.022
Prompt Grounding	0.996	0.998	+0.002

Table 7: Comparison of structured similarity and fixed structured similarity across evaluation metrics.

Overall Interpretation. The fixed structured similarity configuration significantly improves coverage while maintaining similar levels of grounding quality and fluency. This suggests that the modifications primarily enhance recall of structured features without introducing meaningful degradation in language quality or contextual consistency.

6 Demo

6.1 Demo Overview

SoulMap is a web-based demo application that allows users to generate a Vietnamese astrology chart and interact with an AI assistant for personalized interpretation. The demo combines a traditional chart generation engine, a modern web interface, and a RAG-based language model pipeline. Instead of directly asking the LLM to answer from its internal knowledge, the system first generates the user's chart, retrieves relevant astrology knowledge, builds a grounded prompt, and then calls Gemini to produce the final response.

The demo is designed to show that the system can support both initial chart reading and follow-up question answering. The initial reading gives users a general overview of their chart, while the follow-up mode allows them to ask specific questions about study, career, finance, relationships, or personal orientation.

6.2 Demo Objectives

The demo aims to illustrate the following capabilities:

- receiving user information such as name, gender, date of birth, and birth time;
- generating a personalized astrology chart with twelve houses;
- placing main stars, auxiliary stars, and important chart attributes;
- visualizing the generated chart in a web interface;
- providing a star lookup function for exploring the meanings of main stars and auxiliary stars;
- generating an initial AI interpretation;
- allowing users to ask follow-up questions about the chart;
- retrieving relevant knowledge with RAG before calling Gemini.

6.3 System Architecture

The demo consists of three main components:

1. **Frontend**: provides the user interface for entering birth information, viewing the generated chart, and chatting with the AI assistant.
2. **Server-side logic**: handles intermediate processing such as chart generation, temporary data management, and communication with the RAG backend.
3. **Python RAG service**: retrieves relevant documents, constructs prompts, and calls Gemini to generate the final answer.

This architecture separates symbolic chart generation from language generation. The chart engine produces structured user-specific data, while the RAG service provides external knowledge and grounding for the LLM.

6.4 Technologies Used

Table 8 summarizes the main technologies used in the demo.

6.5 Chart Generation Functionality

When the user submits their birth information, the system generates a complete astrology chart. The chart generation process includes converting the input date into lunar information, determining the `Cung_menh` and `Cung_than`, computing the destiny element and configuration, creating the twelve houses, and placing main and auxiliary stars.

Component	Technology
Frontend	React, TypeScript, Vite (Meta Open Source, 2026; Vite Team, 2026)
Routing	TanStack Router
UI	shadcn/ui, Tailwind CSS, lucide-react
Server logic	TanStack Start Server Functions
Chart engine	TypeScript implementation
AI backend	Python FastAPI service (Ramírez, 2026)
RAG	Hybrid retriever with Vietnamese embeddings (Lewis et al., 2020; Karpukhin et al., 2020)
LLM	Gemini API (Google AI for Developers, 2026)
Client storage	LocalStorage

Table 8: Main technologies used in the SoulMap demo.

Each generated house contains information such as its name, zodiac sign, main stars, auxiliary stars, brightness levels, life-cycle stage, and major period. This structured chart is then used in two ways. First, it is displayed visually in the web interface. Second, it is converted into a compact textual context for retrieval and prompt construction.

6.6 AI Interpretation Functionality

The AI assistant supports two interaction modes:

- **Initial reading**: after the chart is generated, the system retrieves important interpretation chunks and asks Gemini to produce an overview of the user's main tendencies.
- **Follow-up question answering**: when the user asks a specific question, the system retrieves new relevant context based on the question and the user's chart, then generates a focused answer.

In both modes, the Python RAG service receives the chart data, builds a chart index, retrieves relevant documents, constructs the prompt, calls Gemini, and returns the generated answer to the frontend.

6.7 Role of RAG in the Demo

RAG plays a central role in making the AI response more grounded. Before calling Gemini, the system searches the knowledge base for interpretation chunks related to the user's chart or current question. The retrieved chunks are inserted into the prompt as supporting evidence.

This design provides several benefits. It reduces the risk of hallucination, allows the system to use a domain-specific knowledge base, and makes the answer more personalized by considering the actual houses and stars in the user's chart. It also separates astrology knowledge from the LLM itself, making the knowledge base easier to update.

6.8 User Interface

The demo interface includes several main screens:

- a landing page introducing the system;
- a chart creation page where users enter birth information;
- a chart list page for viewing previously created charts;
- a chart detail page that displays the twelve-house chart and AI chat;
- a star lookup page for exploring the meanings of astrology stars.

The chart and chat history are stored on the client side so that users can return to their previous charts in the same browser without requiring mandatory login.

6.9 Demo Flow

Figure 5 illustrates the overall workflow of the demo.

The demo flow of the system includes the following steps:

1. The user enters personal information, including name, gender, date of birth, and birth time.
2. The system generates a personalized astrology chart.
3. The generated chart is displayed in the web interface.
4. The system retrieves important interpretation chunks for the initial summary.
5. Gemini generates an initial personality and life-aspect summary.
6. The user asks a follow-up question.
7. The system retrieves new relevant context based on the question.
8. Gemini generates a new grounded answer.

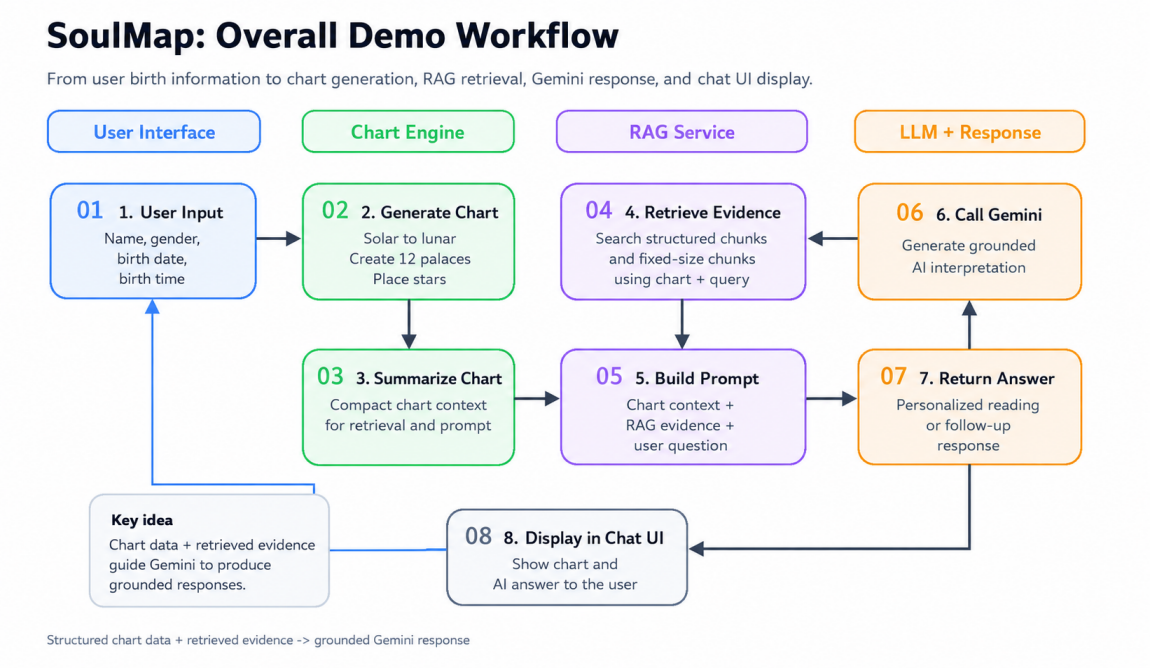

Figure 5: Overall workflow of the SoulMap demo.

6.10 Demo Example

User input:

Full name: Tran Le Ha Dan Gender: Female Solar birth date and time: 2005-07-21T10:20:00 Question: Should I focus more on study or career in the near future?

Expected system behavior:

> The system analyzes the user's astrology chart and retrieves documents related to study, career, the `Cung_menh`, the `Cung_quan_loc`, and other relevant stars. The generated answer should explain the user's tendency, strengths, potential risks, and practical suggestions. The answer should be presented as a reference based on the retrieved context, not as an absolute prediction.

6.11 Summary

The SoulMap demo demonstrates how symbolic astrology chart generation can be combined with a RAG-based AI assistant. The key feature of the demo is that it does not call the LLM directly with only the user's question. Instead, it follows a complete pipeline: chart generation, chart summarization, document retrieval, prompt construction, and Gemini-based response generation. This design enables more personalized and grounded answers while keeping the system extensible for future improvements.

7 Conclusion

This project presents SoulMap, a Vietnamese astrology question answering system based on Retrieval-Augmented Generation. The system combines three main components: a user chart generation module, a knowledge base construction pipeline from PDF documents, and a RAG-based question answering module.

The combination of structured chunking and fixed-size chunking allows the system to use both rule-based astrology knowledge and natural context from the original documents. The hybrid retriever improves the ability to retrieve relevant evidence by combining semantic similarity, structured filtering, and keyword-based matching.

Initial results show that RAG is suitable for this domain because it can reduce hallucination, support knowledge updates, and personalize answers based on the user's chart.

8 Limitation

The current system still has several limitations:

- **PDF quality**: The quality of extracted data depends on the quality of the original PDF documents.
- **OCR and formatting errors**: Some documents may contain extraction errors, especially when the PDF layout is complex.

- **Regex-based parsing**: The current parser relies heavily on regular expressions, so it may not cover all document structures.
- **Lack of benchmark**: The project does not yet have a labeled benchmark dataset for quantitative evaluation.
- **LLM dependency**: The final answer still depends on the reasoning and generation ability of Gemini.
- **Latency and cost**: Calling the LLM and computing embeddings may become expensive when the number of users increases.
- **Interpretive nature of astrology**: Astrology is an interpretive domain, so the system should avoid absolute claims and present answers as references only.

Future Work

Future work includes:

- Completing and cleaning the full structured dataset.
- Building a benchmark dataset with questions, gold evidence, and reference answers.
- Adding a reranker to improve top-k retrieval quality.
- Adding evidence citation to generated answers.
- Optimizing embedding cache and retrieval cache to reduce latency.
- Building a complete user interface for the final demo.

References

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901.

Dich Ly Huyen Co. n.d. *Tu Vi Thuc Hanh*. Nha sach Khai Tri, Sai Gon. Vietnamese astrology manual used as a source document.

Google AI for Developers. 2026. Gemini 2.5 flash. https://ai.google.dev/gemini-api/docs/models/gemini-2.5-flash. Accessed: 2026-05-30.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35.

Meta Open Source. 2026. React documentation. https://react.dev/. Accessed: 2026-05-30.

Nguyen Phat Loc. 1974. *Tu Vi Tong Hop*. Vietnamese astrology reference book.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744.

Sebastián Ramírez. 2026. Fastapi documentation. https://fastapi.tiangolo.com/. Accessed: 2026-05-30.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Van Dang Thai Thu Lang. 1956. *Tu Vi Dau So Tan Bien*. Sai Gon. Vietnamese astrology reference book.

Vite Team. 2026. Vite documentation. https://vite.dev/. Accessed: 2026-05-30.